UCS503P – Software Engineering Project

POTLI

On-Demand Luggage Storage Through Trusted Local Businesses

Submitted by:

1024030088 Satyam Tiwari
1024030525 Ishant Mehndiratta
1024030494 Anshaj
1024030498 Aayush Bindal

BE Third Year, Computer Science and Engineering

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Abstract

POTLI is a web application for short-term luggage storage at local shops, hotels, and other businesses. It serves travelers who need to leave their bags between transport connections, accommodation check-in times, or other activities. The system supports finding storage on a list or map, reserving a time slot, and verifying drop-off and pickup through a six-digit one-time password (OTP) or QR code. Storage partners manage listings and capacity, record cash collected at pickup, and owe a 15% platform fee on each completed booking; administrators approve partners, resolve disputes, and record fee settlements.

This report presents the requirements, system design, implementation, and verification of the system. The implementation uses TypeScript and Vite for three role-specific portals (traveler, storage partner, and administrator), Express for the API, Appwrite for authentication and private file storage, and PostgreSQL with Drizzle ORM for application data. Cash on pickup is the only live payment method; online payments, a field pilot, and deployment remain planned. Automated verification comprises 234 backend API tests, 14 frontend unit tests, and 5 end-to-end smoke flows. The primary evaluation metric is Booking Fulfillment Time (BFT), with a pilot target of a median no greater than three minutes; no BFT measurements have been collected yet.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background and Context . . . 1
1.2 Problem Statement . . . 1
1.3 Stakeholders . . . 1
1.4 Project Objectives . . . 2
1.5 Scope and Current Status . . . 2

2 Requirements and System Analysis **5**
2.1 Functional Requirements . . . 5
2.1.1 Traveler Requirements . . . 5
2.1.2 Partner Requirements . . . 5
2.1.3 Administrator Requirements . . . 5
2.2 Non-Functional Requirements . . . 5
2.3 Use-Case Model . . . 6
2.4 Activity Model . . . 7

3 Methodology and Design **9**
3.1 Development Approach . . . 9
3.2 High-Level Architecture . . . 11
3.3 Data Flow Diagrams . . . 11
3.3.1 Level 0: Context . . . 11
3.3.2 Level 1: System Decomposition . . . 12
3.3.3 Level 2: Booking and Handoff . . . 14
3.4 Domain Model . . . 16
3.5 Component Model . . . 17
3.6 Behavioral Models . . . 20
3.6.1 Sequence Diagram . . . 20
3.6.2 Collaboration Diagram . . . 22
3.6.3 State Diagram . . . 24

4 Implementation Details **25**
4.1 Frontend Portals . . . 25
4.2 Backend API and Middleware . . . 25
4.3 Database Schema . . . 26
4.4 Booking Lifecycle and Capacity Transaction . . . 26
4.5 Handoff Codes and Photographs . . . 26

4.6 Cash Payments and Platform Fee Ledger 27
4.7 Administration 27
4.8 Security and Privacy Considerations 27

5 Testing and Evaluation **29**
5.1 Testing Methodology 29
5.2 Current Verification Status 29
5.3 Evaluation Metrics 30
5.4 Pilot Plan 30

6 Conclusion **31**
6.1 Summary of Work 31
6.2 Key Findings 31
6.3 Future Work and Recommendations 31
6.4 Final Remarks 32

Bibliography **33**

List of Figures

2.1 POTLI use-case diagram 6
2.2 End-to-end POTLI activity diagram 7

3.1 Project development timeline 10
3.2 Level 0 context data flow diagram 12
3.3 Level 1 system data flow diagram 13
3.4 Level 2 booking and luggage-handoff data flow diagram 15
3.5 POTLI domain class diagram 17
3.6 POTLI implementation component diagram 19
3.7 Booking and handoff sequence diagram 21
3.8 Create-booking collaboration diagram 23
3.9 Booking lifecycle state diagram 24

List of Tables

1.1 Implemented and planned project scope 3

4.1 Implemented security controls 28

5.1 Verification status 29
5.2 Proposed evaluation metrics 30

Chapter 1

Introduction

1.1 Background and Context

Travelers may need temporary luggage storage before hotel check-in, after check-out, or while waiting for a transport connection. Carrying bags during these intervals limits movement and access to other activities. Storage facilities at stations and airports may not be near the traveler's destination, and availability may be difficult to establish in advance. Local businesses offer a possible alternative where suitable storage space is available.

POTLI connects travelers with participating businesses and maintains a record of each storage booking. Partners publish their capacity, opening hours, and prices. The system records both luggage handoffs, and administrators review partner details and disputed bookings. Finding a location is therefore only one part of the proposed service; reservation and handoff records are also required.

1.2 Problem Statement

The project addresses the difficulty of finding suitable short-duration luggage storage near a traveler's destination. An informal arrangement may not provide a booking receipt, an availability check, or a consistent record of custody. The system makes these records available without imposing a lengthy procedure on either the traveler or the storage partner.

The interface must be usable on ordinary smartphones and understandable to business owners without technical training. Intermittent connectivity is a further design consideration, particularly during physical handoffs. The project uses standard web technologies and does not require dedicated storage hardware.

1.3 Stakeholders

- **Traveler:** searches, compares locations, books a slot, presents handoff credentials, tracks status, and rates the experience.
- **Storage partner:** registers a business, publishes capacity and hours, confirms drop-off and pickup, and views earnings.
- **Administrator:** verifies partners, audits bookings and fees, reviews reports, and resolves flagged cases.

- **Payment gateway:** planned external service for UPI collections and refunds; it is not yet integrated, and cash on pickup is used instead.

1.4 Project Objectives

1. Deliver a responsive web interface for traveler discovery, account access, partner onboarding, and role-specific operations.
2. Implement secure role-based authentication and a persistent data model for travelers, storage partners, and administrators.
3. Build a booking lifecycle with availability validation, QR/OTP-based drop-off and pickup, timestamps, and photographic handoff evidence.
4. Instrument the workflow so that median BFT and handoff success can be measured objectively.
5. Record cash collection and the platform fee ledger, provide administrative oversight, and prepare for online payments, repeatable deployment, and later geographic scaling.

1.5 Scope and Current Status

Table 1.1 summarizes the current scope. Items marked planned are modeled in the design but not built.

Capability	Status	Evidence
Traveler, partner, and admin portals	**Implemented**	Role-specific interfaces in `code/frontend`, including partner self sign-up
Authentication, email verification, recovery	**Implemented**	Appwrite sessions and JWTs, verified by `/api/v1/auth/me`; mail via Resend
Search with list and map view	**Implemented**	Listings API and Leaflet/OpenStreetMap map
Booking with capacity check	**Implemented**	Transactional creation directly in `Booked`
QR/OTP handoff and drop-off photos	**Implemented**	Derived codes, five-attempt lockout, private Appwrite bucket
Cash on pickup and 15% fee ledger	**Implemented**	`payments` and `fee_settlements` tables, admin settlement
Ratings, disputes, audit log, user disable	**Implemented**	`ratings`, `disputes`, `audit_log` and admin routes
Partner approval and administration	**Implemented**	Approval workflow, listing flags, booking cancellation
Booking expiry	**Implemented**	Periodic background job
Online payments (UPI, card)	**Planned**	Shown as "coming soon"; `PendingPayment` reserved
Field pilot and BFT measurement	**Planned**	No pilot run yet
Production deployment	**Planned**	Staging and hosting not yet configured

Table 1.1: Implemented and planned project scope

Chapter 2

Requirements and System Analysis

2.1 Functional Requirements

2.1.1 Traveler Requirements

A traveler should be able to search by city, landmark, or proximity; compare price, opening hours, and availability; reserve a time window; receive a booking identifier and QR/OTP; verify both physical handoffs; review current and prior bookings; and submit a post-pickup rating.

2.1.2 Partner Requirements

A partner should be able to create a business profile, submit details for verification, define capacity and operating hours, receive reservations, validate handoff credentials, attach a drop-off photograph, restore capacity after pickup, and review earnings.

2.1.3 Administrator Requirements

An administrator should be able to approve or reject partner applications, inspect users and listings, monitor booking states, audit payment and platform-fee records, review performance summaries, and investigate disputes or suspicious listings.

2.2 Non-Functional Requirements

- **Usability:** short forms, clear role-specific screens, and a mobile-responsive interface.
- **Security:** credentials held by an identity provider, signed expiring tokens, authorization checks, server-side validation, secret management, and protected handoff evidence. Authentication controls should follow established web guidance [OWA25].
- **Reliability:** explicit booking states and idempotent state transitions to avoid duplicate confirmations.
- **Performance:** indexed database queries, pagination, compressed assets, and a median BFT target of at most three minutes.

- **Scalability:** stateless API instances and a database structure that can later support indexed location queries.
- **Maintainability:** versioned endpoints, migration scripts, source-controlled diagram definitions, and clear separation of frontend, API, and data layers.

2.3 Use-Case Model

Figure 2.1 places traveler, partner, and administrator interactions inside one system boundary. It demonstrates that booking depends on discovery and that the trust process continues through both handoffs and administrative oversight.

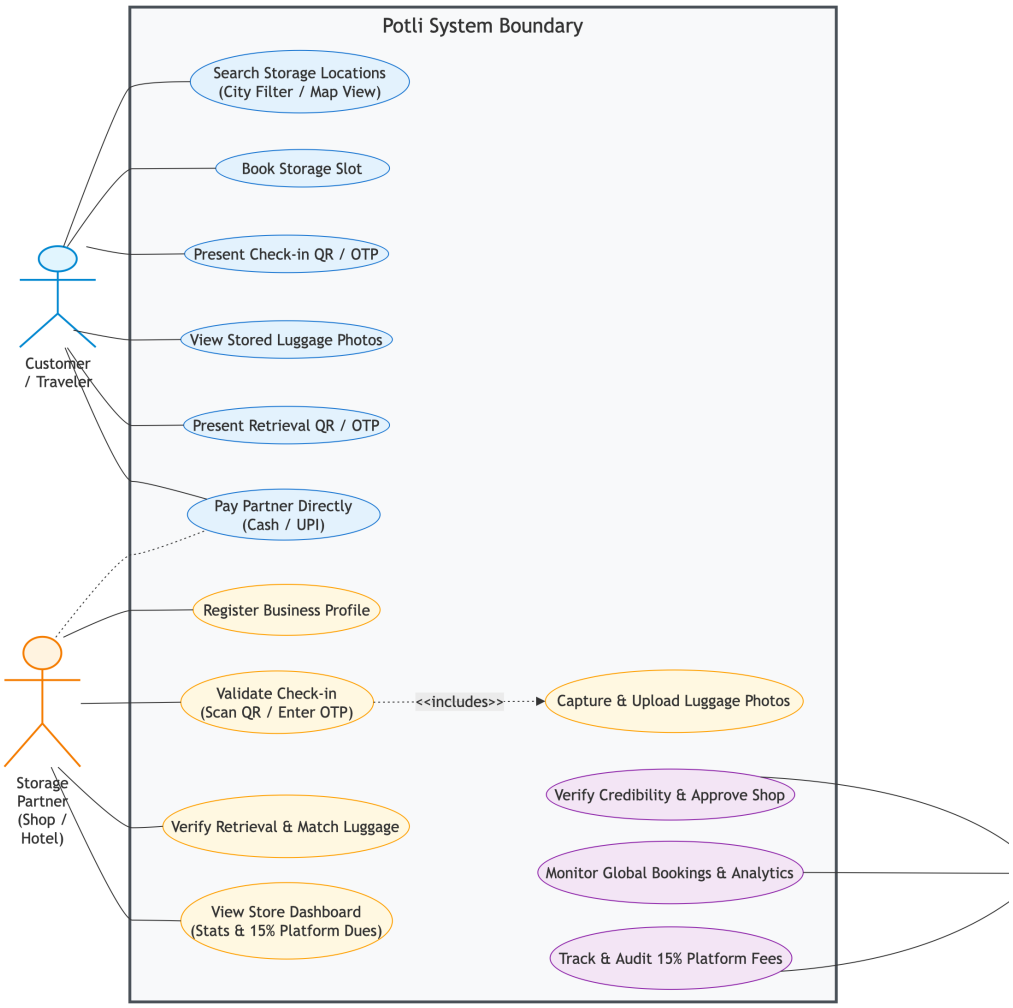


Figure 2.1: POTLI use-case diagram

2.4 Activity Model

The activity diagram in Figure 2.2 follows the complete workflow across traveler, platform, storage partner, and administrator swimlanes. It includes partner approval, discovery, reservation, verification, storage, retrieval, and completion. The model also shows online-payment concerns that are not yet built; the implemented system uses cash on pickup.

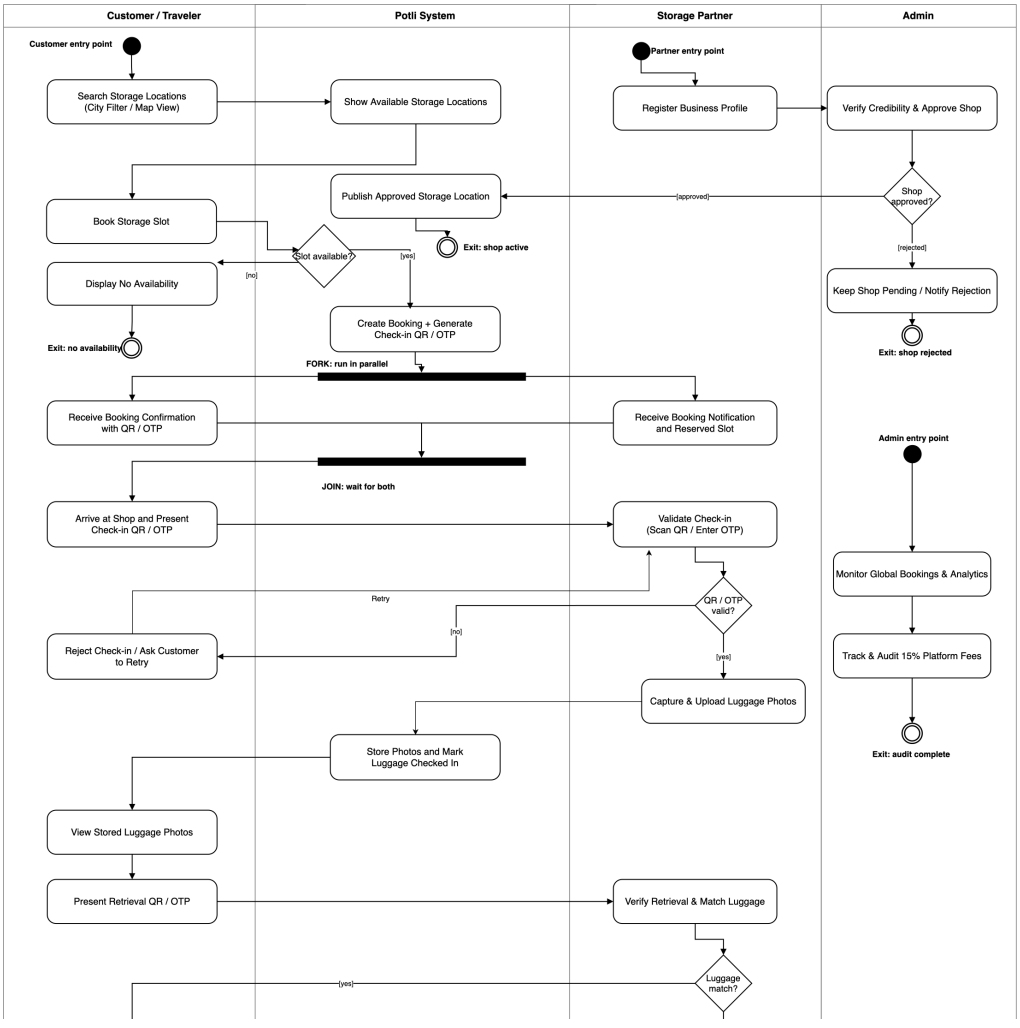


Figure 2.2: End-to-end POTLI activity diagram

Chapter 3

Methodology and Design

3.1 Development Approach

Development was organized into increments: the frontend, authentication API, and profile database first, then listings and search, reservations and luggage handoffs, partner and administrator operations, and automated testing. Online payments, the pilot, and deployment remain.

Figure 3.1 presents the approved development schedule from requirements analysis to the launch demonstration. It records planned activities, not their completion status. The image-upload activity is labeled S3 in the chart; the implementation uses Appwrite Storage.

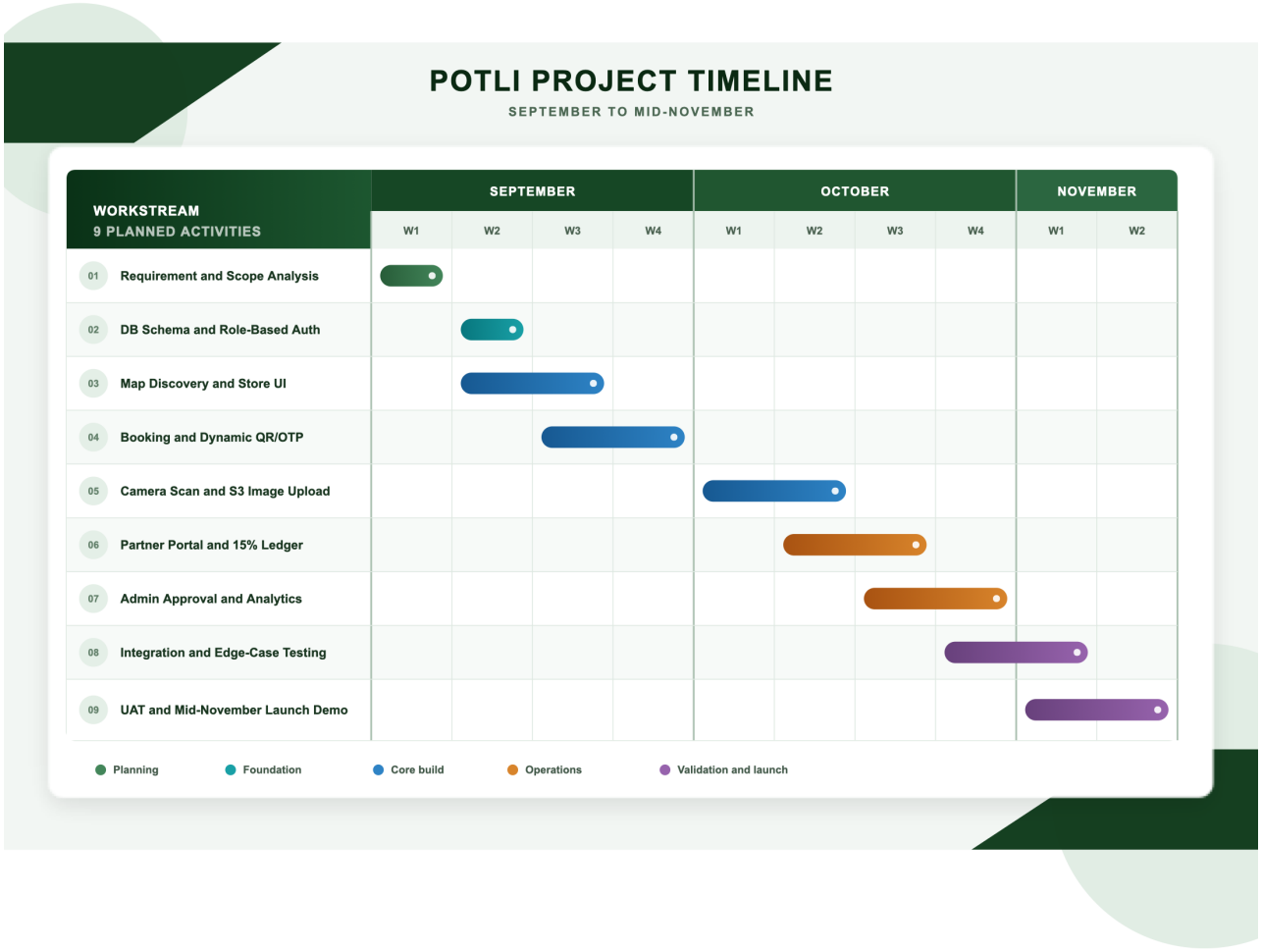


Figure 3.1: Project development timeline

3.2 High-Level Architecture

POTLI uses a three-tier web architecture with a shared domain module:

1. **Presentation layer:** a TypeScript application bundled with Vite [Vit25] that provides traveler, partner, and administrator portals. Maps use Leaflet with OpenStreetMap tiles, and sign-in uses Appwrite's Web SDK.

2. **Application layer:** an Express [Ope25] API on Node.js under `/api/v1`. HTTP middleware provides authentication and role guards, request validation, rate limiting, and upload handling. Routers serve traveler, partner, administrator, file, and system endpoints. A handoff-code service derives OTPs and QR payloads, and a background job expires overdue bookings.

3. **Data layer:** PostgreSQL [Pos25], accessed through a shared Drizzle ORM [Dri] workspace and the `pg` driver. Appwrite [App] manages identity, credentials, email verification, and private file buckets for drop-off and listing photos.

4. **Shared domain module:** `code/shared` holds booking statuses and allowed transitions, pricing and fee calculation, and API types, used by both frontend and backend so that rules are defined once.

The browser signs in to Appwrite and requests a JSON Web Token (JWT) [JBS15] for the current session. It sends the token to the backend using the Bearer scheme. The backend asks Appwrite to identify the account and loads the application profile from PostgreSQL; an account without a profile receives a default traveler profile. Photos are uploaded to the backend, validated, and stored in Appwrite buckets; handoff photos are served only through an authorized backend route.

3.3 Data Flow Diagrams

3.3.1 Level 0: Context

The Level 0 DFD (Figure 3.2) treats POTLI as a single process. Registration, search, booking, business updates, administration, and payment data cross the system boundary through four external entities; the payment gateway is planned for online payments.

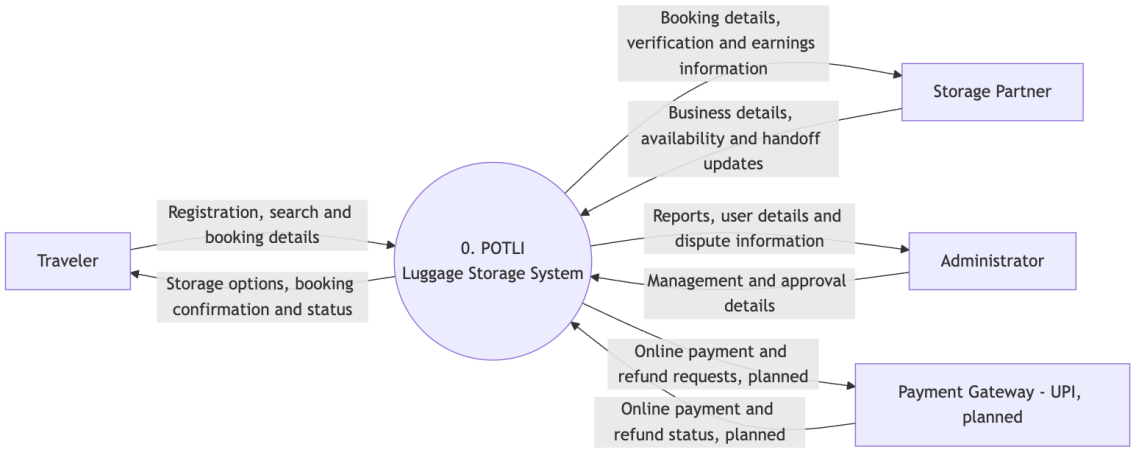


Figure 3.2: Level 0 context data flow diagram

3.3.2 Level 1: System Decomposition

Figure 3.3 decomposes the platform into account management (1.0), storage listing management (2.0), bookings and handoffs (3.0), payment processing (4.0), and platform administration (5.0). Four conceptual stores hold user/partner data, listings, bookings, and payment and fee records. Process 4.0 currently covers cash collection and the platform fee ledger.

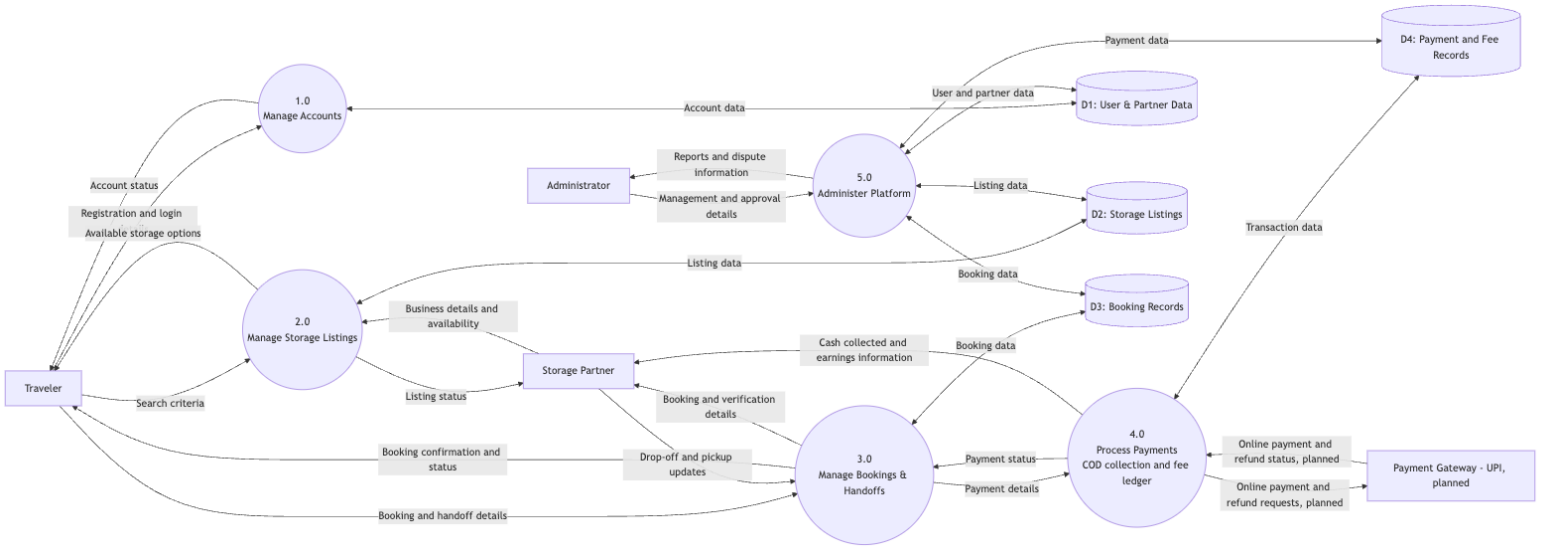


Figure 3.3: Level 1 system data flow diagram

3.3.3 Level 2: Booking and Handoff

The detailed DFD in Figure 3.4 expands process 3.0. A request is validated against listing data, a cash-on-pickup booking is created directly in `Booked`, the drop-off is verified and recorded, and pickup completes the booking while restoring capacity.

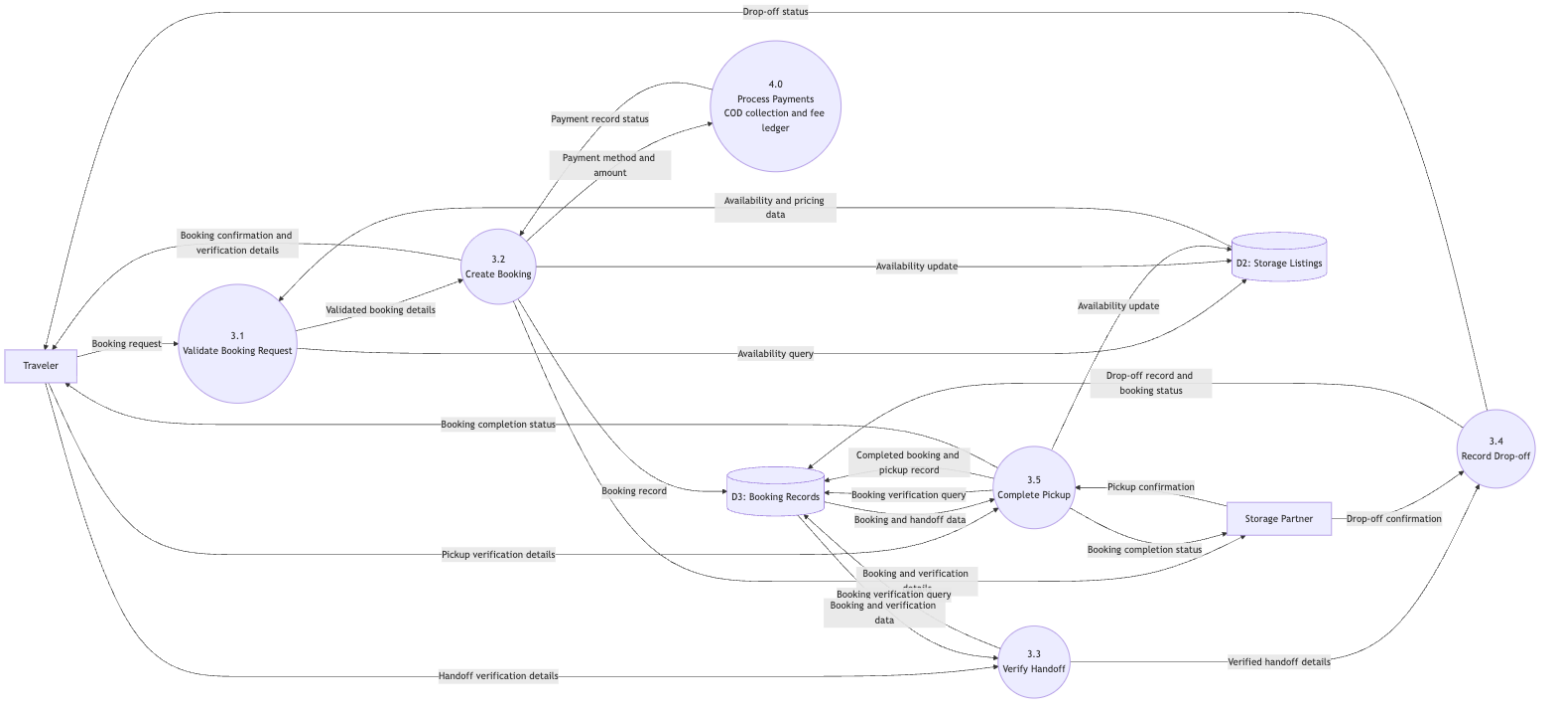


Figure 3.4: Level 2 booking and luggage-handoff data flow diagram

3.4 Domain Model

The class model in Figure 3.5 connects the common `User` identity to traveler, partner, and administrator roles. It separates a partner's business profile from storage listings and composes booking records with handoff and payment information. Fee settlements, disputes, and audit records support administration, and ratings remain optional because they occur after successful pickup. Credentials are held by Appwrite, so the model stores no password hash. Enumerations match the database.

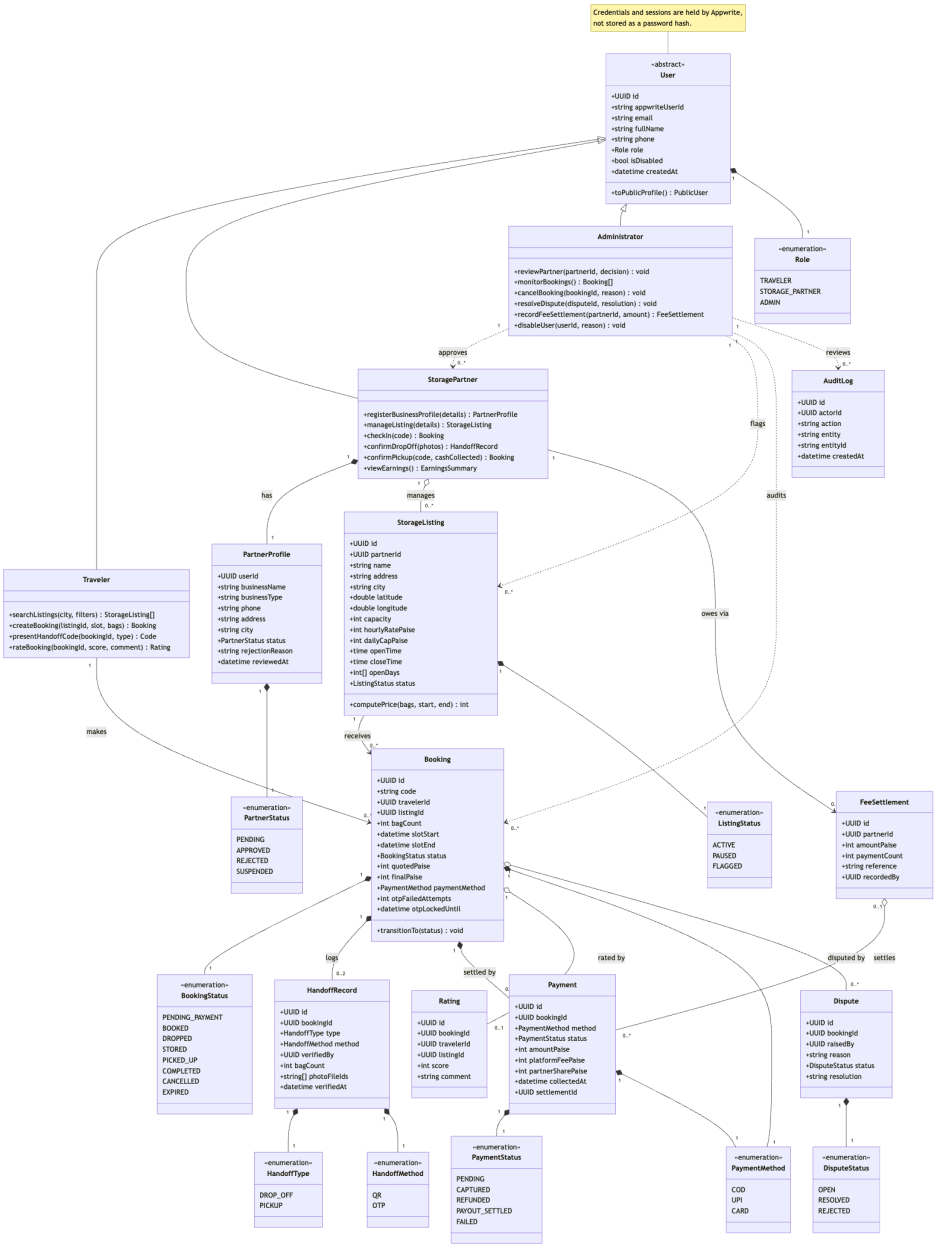


Figure 3.5: POTLI domain class diagram

3.5 Component Model

The component diagram in Figure 3.6 describes the implementation. The browser application uses the backend REST API, the shared domain module, and Appwrite's Web SDK. The backend applies HTTP middleware, mounts the traveler, partner, and

admin routers, runs the expiry job, and uses the handoff code service. An identity adapter reaches Appwrite Auth and Appwrite Storage, and the database workspace reaches PostgreSQL, which holds ten tables. Dashed arrows indicate dependencies; circles and semicircles identify provided and required interfaces.

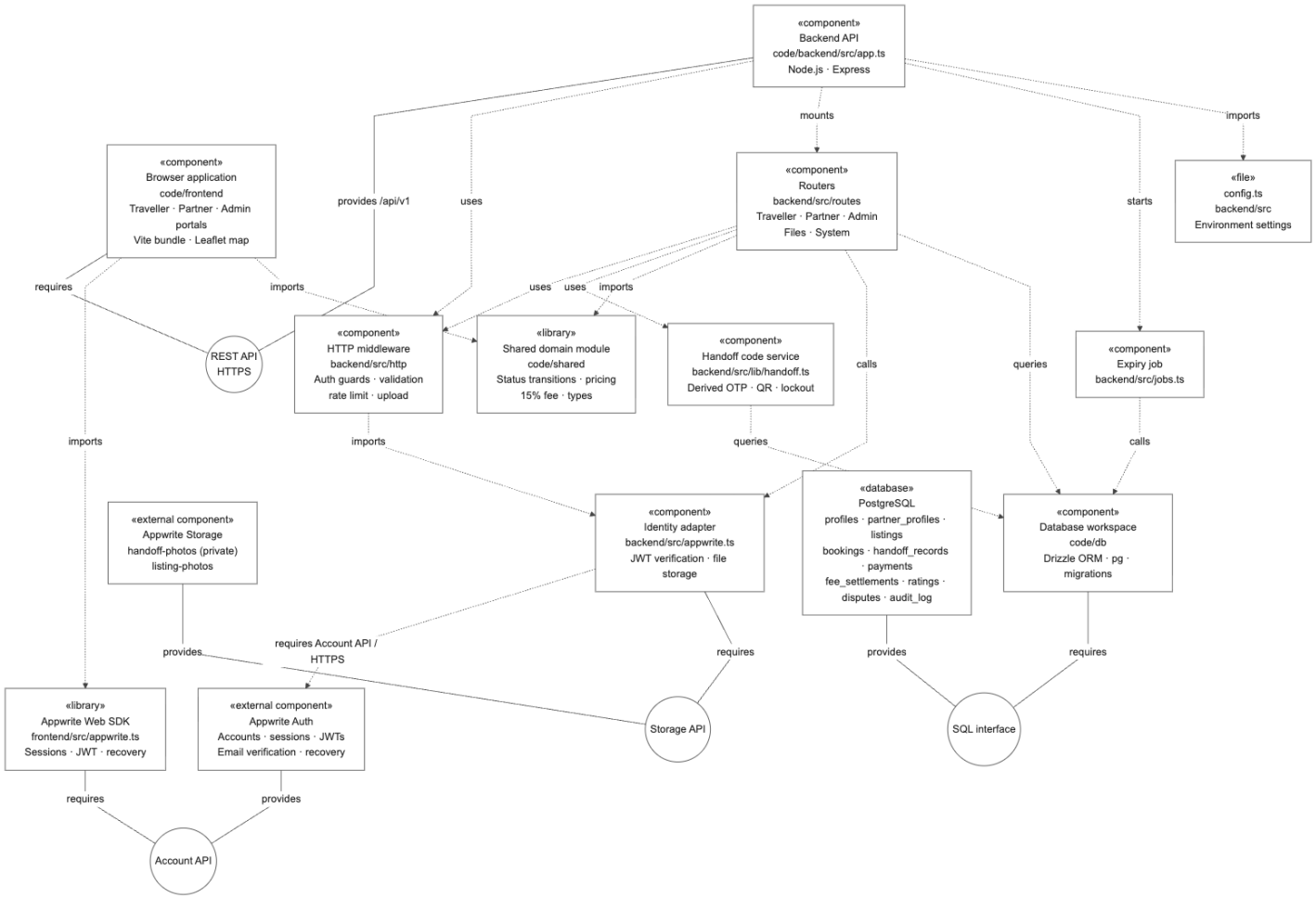


Figure 3.6: POTLI implementation component diagram

3.6 Behavioral Models

3.6.1 Sequence Diagram

Figure 3.7 shows the interaction among the traveler, web application, backend, database, Appwrite Auth and Storage, and partner. It covers discovery, login through an Appwrite session and JWT, cash-on-pickup booking, drop-off with photo upload, pickup with cash collection and fee recording, and rating. The payment gateway is shown as planned only; nothing is charged at booking.

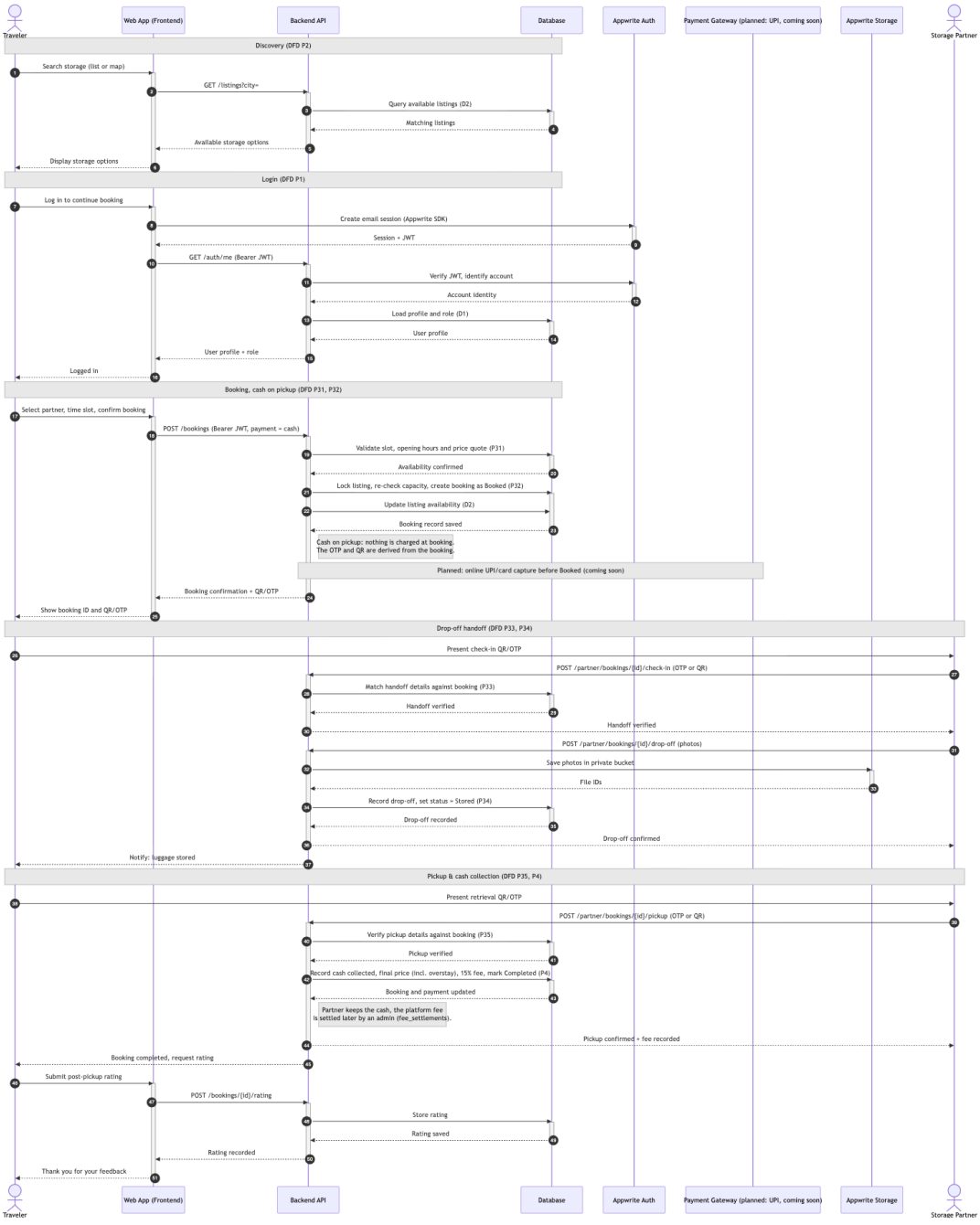


Figure 3.7: Booking and handoff sequence diagram

3.6.2 Collaboration Diagram

The collaboration diagram (Figure 3.8) presents the booking phase as numbered object-to-object messages rather than a time axis. It covers search, login, and cash-on-pickup booking, and emphasizes the links among the web app, identity provider, booking service, listing store, booking store, and partner; the payment gateway appears as a planned dependency.

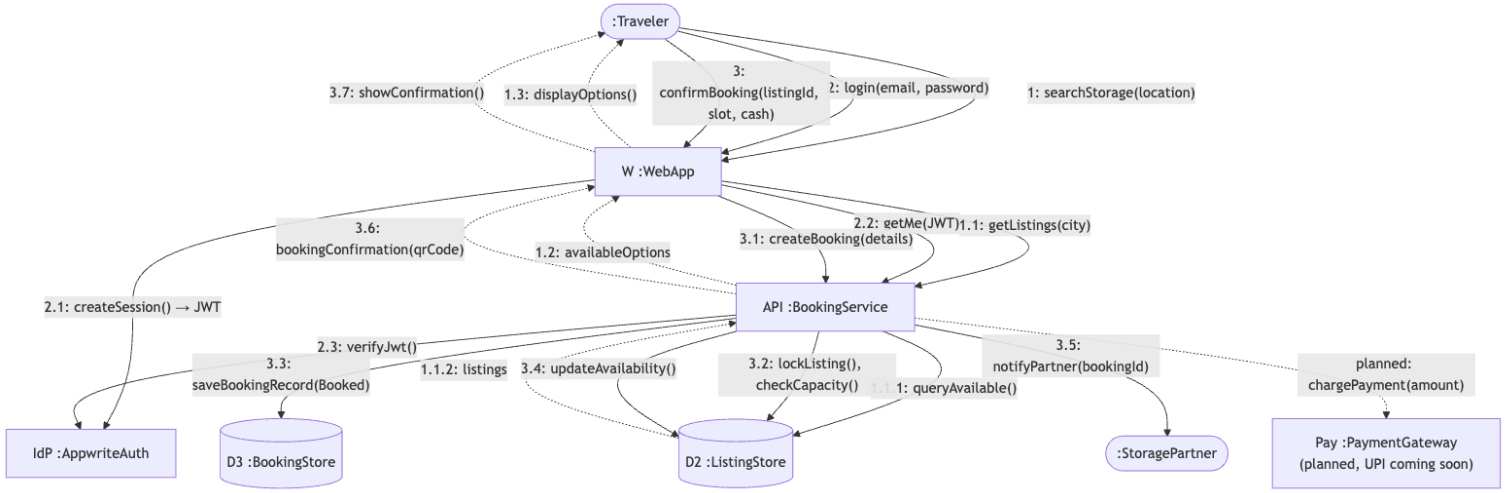


Figure 3.8: Create-booking collaboration diagram

3.6.3 State Diagram

The booking state machine in Figure 3.9 makes valid transitions explicit. A cash-on-pickup booking is created directly in `Booked`; `PendingPayment` is reserved for future online payments. Verified handoffs lead through `Dropped`, `Stored`, and `PickedUp`; recording the cash collected and the 15% fee produces `Completed`. A booking may be cancelled or may expire if no drop-off occurs, and administrators may cancel `Dropped` or `Stored` bookings during disputes. Refunds apply only to future online payments.

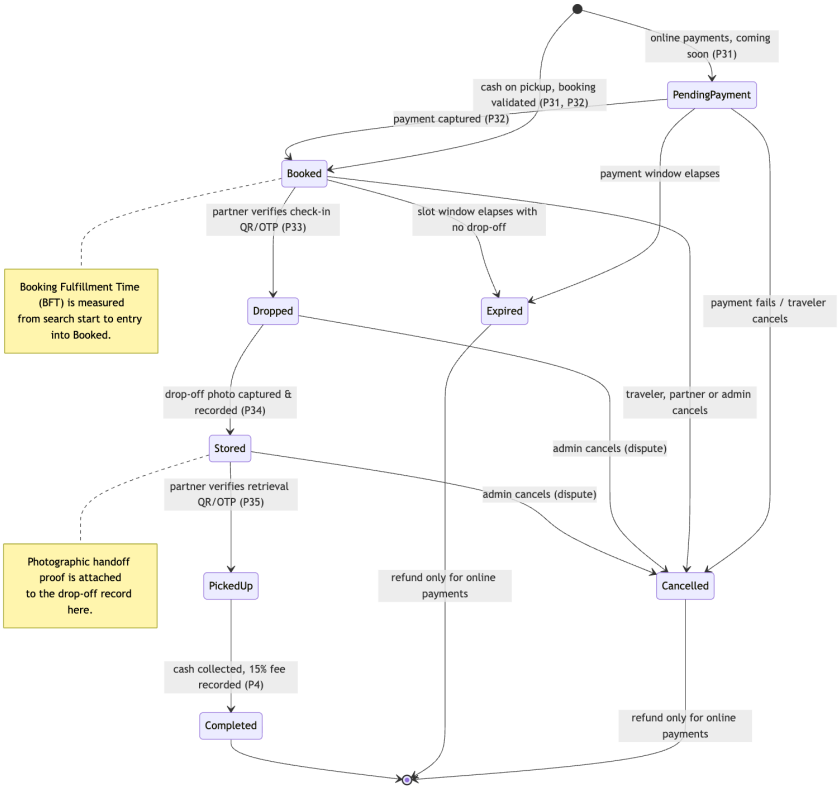


Figure 3.9: Booking lifecycle state diagram

Chapter 4

Implementation Details

4.1 Frontend Portals

The frontend is written in TypeScript, bundled with Vite, and styled with a responsive stylesheet. A hash-based router selects pages for three portals. Travelers search listings by city on a list or Leaflet/OpenStreetMap map, view a price quote, book a slot, see the current OTP and QR code, rate completed bookings, and raise disputes. Partners register through a self sign-up form, manage listings and opening hours, look up bookings, record drop-off with photographs and pickup with cash collected, and view earnings and platform fees owed. Administrators approve or reject partners, moderate listings, cancel bookings, resolve disputes, record fee settlements, review the audit log, and disable users.

Sign-in uses Appwrite's Web SDK to establish a session. The frontend then requests a JWT and calls `/api/v1/auth/me` for the profile and role. Login pages check that the returned role matches the selected portal; this controls navigation only, and every operation is authorized again on the server. Cash on pickup is the only live payment method; UPI and card are shown as "coming soon". Password recovery and email verification use Appwrite, with mail delivered through Resend. The reset page removes the recovery secret from the URL after use.

4.2 Backend API and Middleware

The backend runs on Node.js and Express with Helmet, CORS, a 100 KB JSON limit, and a global rate limiter. Routers are mounted under `/api/v1`:

- **system and auth:** health checks, `POST/auth/login`, and `GET/auth/me`;
- **traveler:** listings, quotes, bookings, cancellation, ratings, and disputes;
- **partner:** application, profile, listings and covers, handoff lookup, check-in, drop-off, pickup, earnings, and fee settlements;
- **admin:** partner review, users, listings, bookings, disputes, ledger and settlements, and audit; and
- **files:** authorized access to stored photographs.

Middleware authenticates the Bearer JWT through Appwrite, loads the profile, and enforces role, verified-email, and approved-partner guards. Request bodies and parameters are validated with schemas. Login is limited to ten attempts per window,

per-user limits protect sensitive routes, and uploads are limited to four images of at most 5 MB, checked by file signature as JPEG, PNG, or WebP. A central error handler hides internal details. A background job runs every five minutes and expires bookings that were never dropped off.

4.3 Database Schema

The Drizzle schema defines ten tables: `profiles`, `partner_profiles`, `listings`, `bookings`, `handoff_records`, `payments`, `fee_settlements`, `ratings`, `disputes`, and `audit_log`. Profiles link to Appwrite accounts by a unique `appwrite_user_id`; credentials are never stored locally. Partner profiles carry an approval status (pending, approved, rejected, or suspended). Listings store location, capacity, hourly rate, daily cap, opening hours and days, and a status (active, paused, or flagged), with check constraints on ranges. Bookings hold a unique code, bag count, slot, status, quoted and final price in paise, payment method, timestamps for each stage, attempt counters, and an idempotency key. Handoff records are unique per booking and type and store the verifying user, bag count, and photograph file identifiers. Payments record method, status, amount, platform fee, partner share, collection details, and an optional settlement. Enumerations for roles, statuses, methods, and handoff types are database types that mirror the shared module. SQL migrations are applied in filename order, and a seed script creates development accounts through Appwrite's Users API.

4.4 Booking Lifecycle and Capacity Transaction

Allowed status transitions are defined once in the shared module and enforced by atomic updates, so a stale or repeated request cannot move a booking twice. Booking creation runs in one database transaction: the listing row is locked, overlapping bookings that still occupy shelf space are counted (overstaying bags are counted until now), and the booking is inserted only if the requested bags fit. An idempotency key makes a retried request return the original booking, and the traveler is given a unique booking code. The price is quoted from the hourly rate with a daily cap and checked against opening hours.

4.5 Handoff Codes and Photographs

Each handoff stage has a six-digit OTP derived from a server secret, the booking identifier, and the stage (drop-off or pickup) using HMAC-SHA-256; nothing is stored, and comparison is constant-time. The QR payload encodes the booking code, stage,

and OTP. Travelers see only the code for the current stage. Five consecutive failed attempts lock the booking's code entry for 15 minutes. At drop-off the partner uploads photographs, which are validated and stored in a private Appwrite Storage bucket; only the file identifiers are stored in the handoff record. Listing photographs use a separate bucket.

4.6 Cash Payments and Platform Fee Ledger

Nothing is charged at booking. At pickup the partner records the cash collected; the final price includes any overstay beyond the booked slot, and the platform fee of 15% is calculated and recorded on the payment together with the partner's share. Because the partner receives the cash directly, partners owe platform fees to POTLI. Administrators group unsettled completed payments into a fee settlement with a reference, and partners can view their dues and settlement history. Online payment methods remain disabled until a gateway is integrated.

4.7 Administration

Administrators review partner applications (approval is required before a partner can publish listings), flag listings, cancel held bookings, and resolve disputes raised by travelers or partners. Cancelling, approvals, fee settlements, user disabling, and booking expiry write entries to an append-only audit log containing the actor, entity, metadata, and IP address.

4.8 Security and Privacy Considerations

Controls now implemented are summarized in Table 4.1. Authentication follows established guidance [OWA25].

Area	**Control**
Identity	Credentials, sessions, JWTs, and email verification held by Appwrite
Authorization	Server-side role, verified-email, and approved-partner guards
Input	Schema validation, body size limit, image signature and size checks
Abuse	Global, login, and per-user rate limits; five-attempt OTP lockout
Handoff evidence	Private photo bucket served through an authorized route
Integrity	Atomic transitions, capacity lock, idempotency keys, audit log
Transport headers	Helmet headers and restricted CORS configuration

Table 4.1: Implemented security controls

Remaining work includes production secret management, retention periods for photographs and personal data, a review of rate-limit and recovery-link handling in a deployed environment, and a security assessment before public launch. Online payment integration will add its own requirements.

Chapter 5

Testing and Evaluation

5.1 Testing Methodology

Verification combines:

- **Backend API tests** against an isolated PostgreSQL database covering authorization, booking races, handoff codes, payments, and invalid transitions;
- **Frontend unit tests** for shared utilities;
- **End-to-end smoke tests** that drive the portals in a browser; and
- **Pilot user testing** with representative travelers and local businesses, which has not yet been run.

5.2 Current Verification Status

Table 5.1 reports the automated checks implemented in the repository. Live authentication and recovery-email checks additionally require Appwrite and a configured mail service. No performance tests or pilot measurements have been carried out, so no BFT result is reported.

Check	Status	Scope
Backend API tests	234 tests	Routes, authorization, booking capacity, handoff, payments, administration
Frontend unit tests	14 tests	Shared frontend utilities
End-to-end smoke flows	5 flows (Playwright)	Core portal journeys in a browser
TypeScript compilation and Vite build	Available	Type and build checks
Performance and concurrency load tests	Not yet run	Search and booking under load
Pilot and BFT measurements	Not yet collected	Requires deployment and field trial

Table 5.1: Verification status

5.3 Evaluation Metrics

Metric	Target	Measurement
Median Booking Fulfillment Time	≤ 3 minutes	Search-start timestamp to confirmed-booking timestamp
Handoff success rate	Establish after pilot	Completed drop-off and pickup without dispute / initiated handoffs
Partner utilization	Establish baseline	Booked capacity / listed capacity per partner-week
Payout turnaround	Define with gateway	Completion timestamp to credited-payout timestamp
Traveler satisfaction	1–5 scale	Post-pickup rating and short qualitative feedback
Business-hour availability	$\geq 99\%$ pilot goal	Successful health/search probes during defined hours

Table 5.2: Proposed evaluation metrics

5.4 Pilot Plan

A small locality near a railway station or busy market can provide a controlled first deployment. The proposed pilot includes 8–12 partner businesses and 15–20 traveler participants. Before use, partners receive a brief handoff procedure and the team confirms business details. Application timestamps provide quantitative measures, while structured post-use questions capture convenience, trust, and onboarding difficulty. Any safety incident or disputed handoff takes priority over performance measurement.

Chapter 6

Conclusion

6.1 Summary of Work

POTLI now provides a working luggage-storage service in three portals: search on a list or map, capacity-checked booking, OTP and QR handoffs with drop-off photographs, cash collection with a 15% platform fee ledger, partner approval, ratings, disputes, an audit log, and administrative settlement. The design diagrams, database schema, and implementation have been aligned, and the system is covered by 234 backend tests, 14 frontend tests, and 5 end-to-end flows.

Online payments, a field pilot, and deployment remain outstanding. The report therefore makes no performance or BFT claims.

6.2 Key Findings

- A booking record retains the information needed to verify custody: handoff credentials, timestamps, and photographic evidence.
- Availability and booking creation form the central consistency boundary and are updated in one locked transaction to prevent overbooking.
- Identity is managed by Appwrite; PostgreSQL stores roles and business records, and shared transition and pricing rules keep frontend and backend consistent.
- With cash on pickup, the platform fee becomes a receivable that needs an explicit ledger and settlement process.
- The BFT target is meaningful only once the system is deployed and measured in a pilot.

6.3 Future Work and Recommendations

1. Integrate a sandbox payment gateway (UPI) and enable `PendingPayment`, refunds, and automated fee collection.
2. Deploy a staging environment with managed secrets and monitoring.
3. Add load tests for nearby search and concurrent booking.

4. Run the proposed pilot and replace target values with measured BFT and handoff success rates.

5. Define retention periods for photographs and personal data, and complete a pre-launch security review.

6. Extend notifications and analytics for partners and administrators.

6.4 Final Remarks

The next milestone is deployment and a pilot with local partners. The existing timestamps and booking states allow BFT and handoff success to be measured directly once real users are involved.

Bibliography

[App] Appwrite. *Appwrite Documentation*. URL: https://appwrite.io/docs (cit. on p. 11).

[Dri] Drizzle Team. *Drizzle ORM Documentation*. URL: https://orm.drizzle.team/docs/overview (cit. on p. 11).

[JBS15] Michael Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Tech. rep. RFC 7519. Internet Engineering Task Force, 2015 (cit. on p. 11).

[Ope25] OpenJS Foundation. *Express: Fast, unopinionated, minimalist web framework for Node.js*. 2025. URL: https://expressjs.com/ (visited on 10/06/2025) (cit. on p. 11).

[OWA25] OWASP Foundation. *Authentication Cheat Sheet*. 2025. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (visited on 10/06/2025) (cit. on pp. 5, 27).

[Pos25] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. URL: https://www.postgresql.org/docs/ (visited on 10/06/2025) (cit. on p. 11).

[Vit25] Vite Contributors. *Vite: Next Generation Frontend Tooling*. 2025. URL: https://vite.dev/ (visited on 10/06/2025) (cit. on p. 11).